Awareness Gap Miner – Team Guide

Internal handover for Rishi, Sanjai, Abid and Purna. Goal: anyone reading this can explain, run and defend the whole project. Repo root: **NLP/**. Deeper detail lives in **docs/** (ARCHITECTURE, NLP_PIPELINE, DATA_COLLECTION, LIMITATIONS, OPERATIONS, evaluation/EVALUATION.md).

1. What the project is

A platform that collects public text about Indian government welfare schemes (PM-KISAN, PMAY, Ayushman Bharat, MGNREGA etc. – 15 schemes), runs it through NLP, and shows **awareness-gap signals**: questions/complaints/comments showing people lack or misunderstand scheme information. A plain question is NOT a gap (e.g. "How do I apply?" is no gap; "I never got my money and nobody told me why" is a gap). Results describe the analysed text only, not population-wide awareness – say this in any presentation.

2. Architecture in one picture

Part	Tech	Job
Frontend (port 3000)	Next.js 16, React 19, Tailwind 4	Dashboards, records, clusters, geography, live monitor, text analyzer, models page. Filters live in the URL.
API (port 8000)	FastAPI, SQLAlchemy 2, Alembic	REST + Server-Sent Events. API-key auth, rate limit, upload limits, audit trail.
Worker	python -m backend.worker	Runs collection jobs, picks up pending NLP jobs, heartbeat, resumes stalled jobs.
NLP	sentence-transformers, scikit-learn, HDBSCAN	Scheme detection, language, intent, awareness gap, gap type, evidence, clustering, search.
Database	SQLite (dev) / PostgreSQL (Docker)	documents, sources, campaigns, crawl pages, clusters, model versions. All times stored UTC.

One pipeline for everything: dataset uploads and live collection both go through *services/pipeline.py* (validate -> normalise -> redact PII -> dedup -> embed -> NLP -> store). The only difference is *origin* = dataset or live, which keeps the two workspaces separate in analytics.

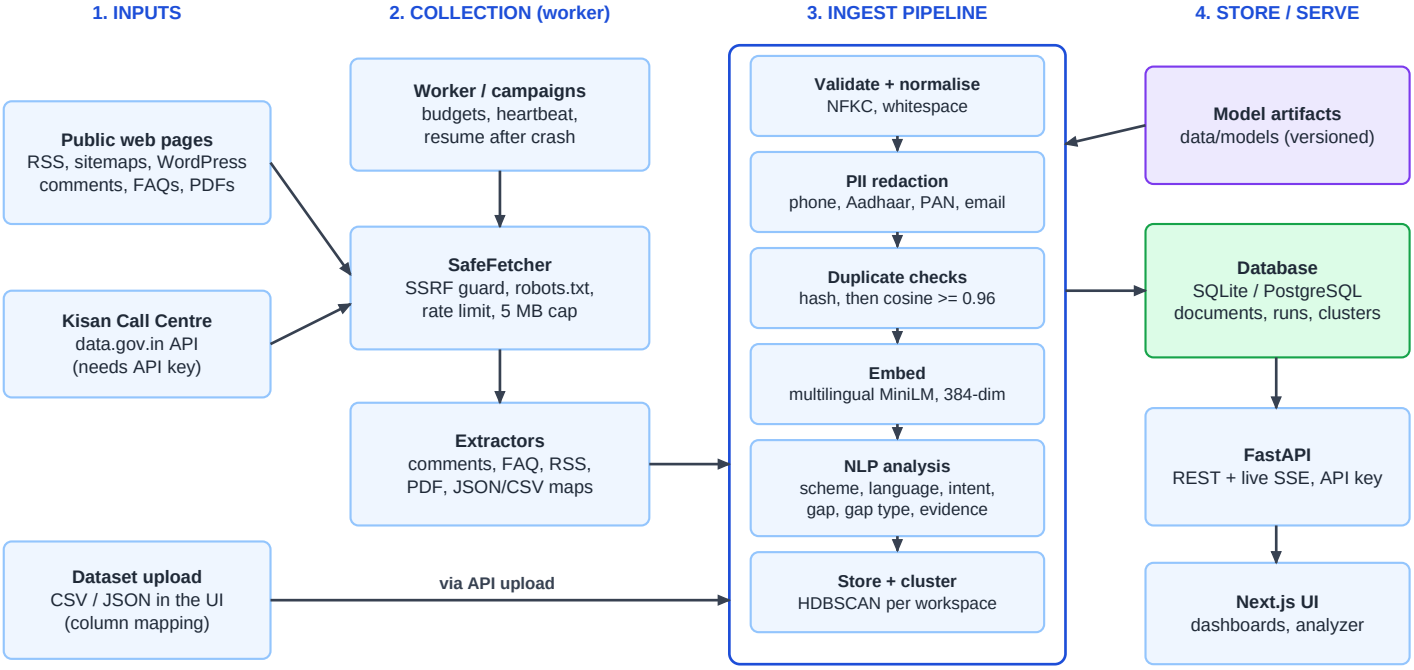


Fig. 1: end-to-end architecture. Dataset uploads skip the collection stage; both paths share the same ingest pipeline and database.

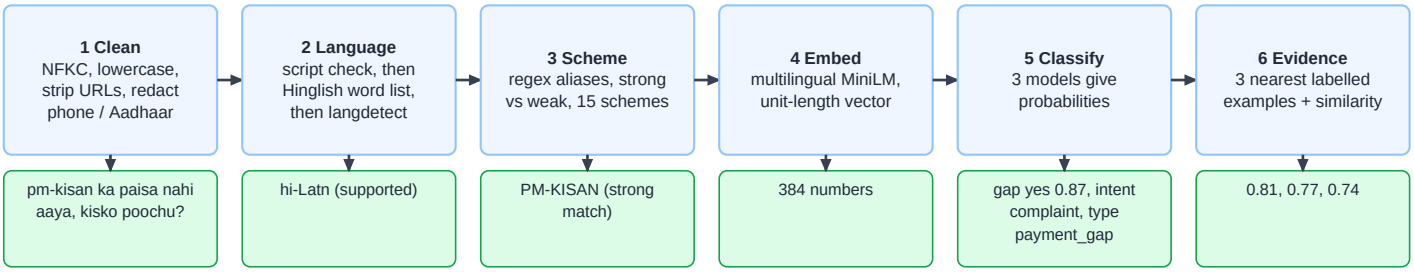
3. Data collection (backend/app/collection/)

- **Registry** (registry.py): curated sources, each with a policy (allowed / credentials_required / unsupported / manual_review / blocked / paused). Verification status changes only from real runs.
- **Safe fetching** (safe_http.py): http/https only, public IPs only (SSRF + DNS-rebinding defence), redirects re-checked, 5 MB cap, robots.txt per RFC 9309, per-host delay, retries with backoff. TLS is never disabled; broken government certificates are reported as failures.
- **Extraction** (extractors.py): WordPress comments (each with its own timestamp), FAQs, RSS/sitemaps, PDFs, JSON/CSV APIs. A record is kept only if it names a scheme.
- **Timestamps**: source_published_at comes only from the source (else empty, shown as "Publication date unavailable"); first_seen_at / collected_at / processed_at are system times.
- **Kisan Call Centre (KCC)**: official data.gov.in API, resource cef25fe2-..., filtered to QueryType=Government Schemes. Needs **DATA_GOV_IN_API_KEY** in .env and a **worker restart** after editing it. Each call is one record, deduplicated by KCCCallID (not text). Gives real state/district/date. Last job: 195 records.
- Geography is stored only when a source states it (KCC, dataset). Nothing is guessed, so web comments show no location.

4. NLP techniques, explained

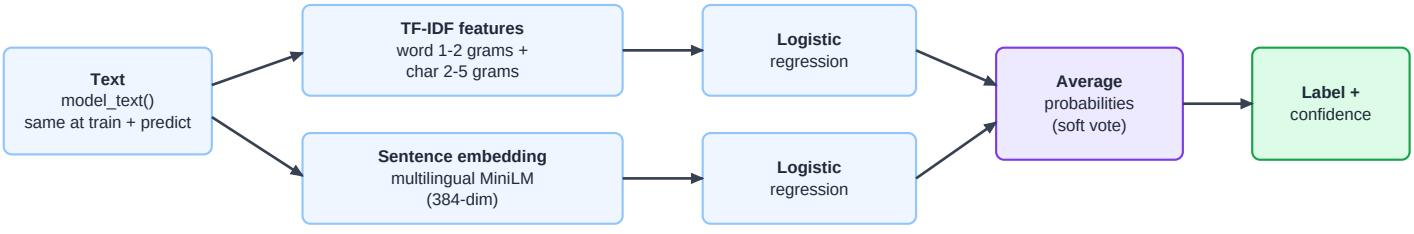
Everything is in **backend/app/nlp/**. The same text goes through the same steps at training time and at prediction time (model_text() guarantees this), so the model never sees differently prepared input.

Worked example (illustrative values): one stored comment



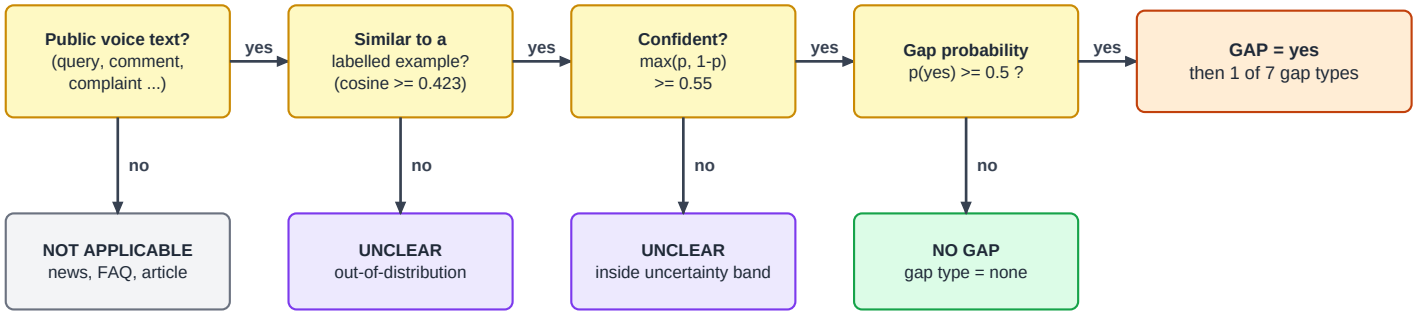
Technique	What it does (plain words)	Why we need it	File
Unicode normalisation (NFKC) + cleaning	Makes look-alike characters identical, removes zero-width chars and extra spaces; lowercases and drops URLs for the model.	Hindi text from the web has many invisible variants; without this the same sentence hashes and embeds differently.	text.py
PII redaction (regex)	Replaces phones, Aadhaar, PAN, e-mail, PIN codes and 9-18 digit IDs with tags before anything is stored.	Privacy. Names are not reliably detectable and may remain.	text.py
Language detection (3 layers)	1) Unicode script share (Devanagari = Hindi). 2) Word lists for romanized Hindi/Marathi. 3) langdetect, rejected if implausible for India.	langdetect calls Hinglish 'Indonesian' or 'Swahili'; the word list fixes that. Supported = en, hi, hi-Latn.	text.py
Rule-based scheme detection	Regex over 15 scheme aliases (strong aliases like 'PM-KISAN', weak like 'kisan scheme'); spaces and hyphens are interchangeable; word boundaries avoid false hits.	Transparent, no training data needed. Unknown state schemes become Other/Unknown.	schemes.py
TF-IDF n-grams	Counts word pairs and 2-5 letter chunks, down-weighting common words. Char chunks survive spelling variants (paisa / paise / paysa).	Strong, cheap baseline; also gives readable 'top terms' for explanations.	estimators.py
Sentence embeddings	A neural encoder (paraphrase-multilingual-MiniLM-L12-v2) turns a sentence into 384 numbers so similar meaning, even across English / Hindi / Hinglish, lands close together.	Lets one classifier work across languages; powers search, evidence, duplicates and clusters.	embeddings.py
Logistic regression + class balancing	A linear classifier on top of TF-IDF or embeddings; class_weight=balanced so rare labels are not ignored.	Works with a few hundred labels; gives calibrated-ish probabilities we can threshold.	estimators.py
Soft-voting ensemble	Average of the TF-IDF and embedding probabilities.	Won on intent and gap type; embeddings alone won on awareness gap.	estimators.py
Uncertainty band + OOD guard	A gap decision needs confidence >= 0.55 and similarity >= 0.423 to the nearest labelled example; otherwise 'unclear'.	Prevents confident-looking guesses on text unlike the training data.	service.py
Nearest-example evidence	Dot product of unit vectors = cosine similarity; the 3 closest labelled examples are stored with each record.	Every label is traceable to examples a human can check.	service.py
Near-duplicate detection	Exact SHA-256 hash first, then cosine >= 0.96 between embeddings (brute-force vector index).	Stops the same comment or call-centre summary being counted twice (KCC is deduplicated by call id instead).	pipeline.py
HDBSCAN clustering	PCA to 20 dims, then density clustering (leaf selection) finds topic groups without choosing their number; c-TF-IDF picks keywords per cluster.	Shows recurring complaint themes; points that fit no group are left as noise.	clustering.py
Semantic search	Embeds the query and returns records above cosine 0.35.	Finds records by meaning, not exact words.	vector_index.py

Classifier design (estimators.py) and selection (training.py)



Candidates compared per task: majority baseline, TF-IDF LR, embedding LR, ensemble.

Awareness-gap decision rules (service.py _gap)



- **Reading the decision chart.** The 0.55 band was chosen on the validation set (keeps at least 85% of records decided). Hindi and Hinglish are classified like English: a record is never unclear because of its language alone.
- **Labels.** Awareness gap: yes / no (plus unclear, not_applicable decided by the rules above). Intent (8): application, general_information, complaint, eligibility, benefits, deadline, payment, documentation. Gap type (7): application, eligibility, benefit, deadline, payment, documentation gap, misconception; 'none' when there is no gap. Intent and gap type always get a label; weak ones are flagged low_confidence.
- **Training protocol** (training.py): fit each candidate on train, pick the best on validation macro-F1, choose the uncertainty band on validation, refit on train+val, and evaluate **once** on the test split, overall and per slice. A record and its translations stay in the same split, and live comments are grouped by source page, so there is no leakage. Grouped cross-validation is a second check.
- **Corpus**: 127 labelled English records (29 synthetic augmentations) + 126 Hindi + 127 romanized-Hindi translations + 252 live comments. **Translations and live labels were produced by an AI assistant** (translations inherit the original labels); disclose this. Rare classes (payment, deadline, documentation) have only a handful of examples.
- **Why the multilingual model.** The first English-only model mapped nearly all Devanagari text to similar vectors, so Hindi came out 'unclear'. Switching models and adding Hindi/Hinglish examples fixed it; unclear in the live data fell from 2,703 to 482 records.
- **Operator corrections** (label_source manual/review) always override the model and survive reprocessing; the model prediction is kept separately in predicted_gap.

Test macro-F1 (101 test records)	Gap	Intent	Gap type
Selected model	0.65 (embedding LogReg)	0.43 (ensemble)	0.35 (ensemble)
Majority baseline	0.34	0.05	0.08
Grouped cross-validation	0.60	0.34	0.28
Gap by slice	English 0.53 / Hindi 0.74 / Hinglish 0.56 / live 0.61		

Honest reading: gap detection is decent; intent and gap type are weak because data is small and rare classes have few examples. Slices have only 12-38 test records, so one record moves a score by several points. Macro-F1 averages the score over labels so rare labels count equally.

5. Current numbers (live workspace)

3,329 unique records (620 duplicates excluded): 385 gap signals, 2,170 no gap, 482 unclear (down from 2,703), 292 not applicable; gap rate 15% over 2,555 decided records. Dataset workspace: 98 records, 63 gap / 35 no gap. 133 backend tests pass; frontend lint, typecheck and build are clean; browser check of all 20 pages shows no errors.

6. How to run it

Step	Command
Setup once	python -m venv venv; venv\Scripts\activate; pip install -r requirements.txt; copy .env.example .env; npm install; npm --prefix frontend install
Set in .env	ADMIN_API_KEY (any long random string; also what you type under "Operator" in the UI), DATA_GOV_IN_API_KEY (free from data.gov.in). Never commit .env or share the keys.
Database + data	npm run db:migrate; npm run seed
Start everything	npm run dev (UI 3000 + API 8000 + worker). Close heavy apps first; first analyzer call loads the model.
Other	npm test (pytest) | npm run test:frontend | npm run train (retrain + evaluation report) | npm run reprocess | npm run backup
Docker	docker compose up --build (PostgreSQL 16, migrate, api, worker, web). Written but never run, because Docker was not running.

Use the venv Python (venv\Scripts\python.exe), not the system Python, otherwise imports like pydantic_settings fail. To collect: Operator access -> Collection jobs -> start a job. To label data yourself: Records -> open a record -> correct labels.

7. Where things are

Path	Contents
backend/app/routers/	API endpoints (analytics, documents, datasets, collection, sources, clusters, analyze, models_api, schemes, system)
backend/app/services/	pipeline.py (ingest + NLP), analytics.py, clustering, dataset import, vector index
backend/app/collection/	engine.py (campaigns), registry.py (sources), safe_http.py, robots.py, extractors.py
backend/app/nlp/	text.py, schemes.py, geo.py, estimators.py, training.py, service.py (runtime), artifacts.py, embeddings.py
backend/migrations/, backend/worker.py	Alembic schema history; worker process
frontend/src/app, src/components	Pages (thin server components) and client views, ui.tsx primitives, charts.tsx
data/raw/, data/models/	Labelled training CSVs; versioned trained models + reference set
tests/	pytest suite (mock HTTP only, no external requests)

8. Roles for the four of us

Rishi built all of it, so each person owns one area: reads the listed files, can demo it, and answers viva questions on it.

Person	Owns	Must be able to explain	Read first
Rishi (lead)	Overall architecture, integration, final demo, git/release	End-to-end flow, pipeline, security choices, every limitation, how to restart/fix a stuck run	README, ARCHITECTURE.md, services/pipeline.py, main.py
Sanjai	Data collection and backend API	SSRF defences, robots/politeness, source policies, KCC integration and API key, timestamps, job/worker/SSE flow, failed-source reasons	DATA_COLLECTION.md, collection/*, worker.py, routers/collection.py
Abid	Frontend and UX	Each page and what its numbers mean (denominators, gap rate, unclear), URL-synced filters, live monitor, analyzer, build/lint, screenshots	frontend/src/app, components/, docs/screenshots
Purna	NLP, models and evaluation	Corpus and annotation, why multilingual model, candidates, thresholds/OOD guard, metrics and their weakness, AI-label disclosure, retraining	NLP_PIPELINE.md, LIMITATIONS.md, EVALUATION.md, nlp/training.py, nlp/service.py

Shared: everyone runs the app once locally, runs npm test, and rehearses the 'Honest answers' below. Suggested split for the report/slides: Sanjai = collection, Purna = NLP + results, Abid = UI + demo, Rishi = intro, architecture, limitations.

9. Honest answers for questions

- **Are labels human?** The original 127 are human-curated (29 synthetic augmentations). Hindi translations and 252 live labels are AI-assisted. No inter-annotator agreement was measured.
- **Why are intent / gap-type scores low?** Small data, many classes, rare classes with few examples. Every record still gets a label; weak ones are flagged low_confidence.
- **Why do some official sites fail?** Incomplete TLS chains, HTTP 403 to bots, timeouts, JavaScript-only pages (no headless browser). We report real failures and never bypass blocks.
- **Other limits:** scheme detection is rule-based (state schemes become Other/Unknown); Marathi/Gujarati etc. classified through the multilingual space with no labelled examples; Postgres/Docker not executed; brute-force vector index; one operator key, no user accounts; names in comments may remain.
- **Security:** operator API key for all writes, no default credential, rate limiting, SSRF protection, PII redaction, audit log. data.gov.in key is added only at request time and redacted from errors.